

C++ OPERATORS

The Beginner Friendly Master Guide

CodeHive Academy

Module 4: Logic & Calculation Foundations

1. Operators (Overview)

Operators are symbols that tell the compiler to perform specific mathematical or logical manipulations. In **CodeHive** terminology, think of them as the "engines of change" that transform static data into dynamic results.

Basic Anatomy:

In the expression $5 + 3$:

- **5 and 3** are **Operands** (the data).
- **+** is the **Operator** (the action).

Core Categories Covered in this Guide:

- **Arithmetic Operators:** For basic and advanced math.
- **Assignment Operators:** For storing and updating data.
- **Comparison Operators:** For testing truths.
- **Logical Operators:** For complex decision making.

2. Arithmetic Operators

Used for fundamental mathematical operations. These are the tools you use for calculations, physics in games, or financial processing.

Operator	Description	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	6 / 3	2
%	Modulus	5 % 3	2
++	Increment	i++	i = i + 1
--	Decrement	i--	i = i - 1

3. Arithmetic Implementation

Let's look at how these operators work inside a real CodeHive source file. Pay close attention to the Modulus (remainder) operator, which is highly useful for checking even/odd numbers.

```
#include <iostream> using namespace std; int main() { int a = 5, b = 3;
cout << "Sum: " << a + b << endl; cout << "Difference: " << a - b << endl;
cout << "Product: " << a * b << endl; cout << "Quotient: " << a / b <<
endl; cout << "Remainder: " << a % b << endl; return 0; }
```

4. Assignment Operators

Assignment operators are used to give values to variables. While `=` is the most common, C++ offers compound assignment operators that save time and space.

Operator	Example	Equivalent Logic
<code>=</code>	<code>x = 5</code>	Assign 5 to x
<code>+=</code>	<code>x += 5</code>	<code>x = x + 5</code>
<code>-=</code>	<code>x -= 5</code>	<code>x = x - 5</code>
<code>*=</code>	<code>x *= 5</code>	<code>x = x * 5</code>
<code>/=</code>	<code>x /= 5</code>	<code>x = x / 5</code>
<code>%=</code>	<code>x %= 5</code>	<code>x = x % 5</code>

5. Compound Assignment Examples

Using compound operators makes your code cleaner and more readable for other CodeHive developers.

```
int x = 10; x += 5; // x is now 15 x *= 2; // x is now 30 x /= 3; // x is now 10
cout << "Final Value of x: " << x << endl;
```

Pro Tip: Compound operators are not just faster to type; they also reduce the risk of repeating the wrong variable name in complex logic.

6. Comparison (Relational) Operators

Comparison operators compare two values. They are essential for creating **Conditions**. These operators always return a `bool` (true or false).

Operator	Example	Meaning
<code>==</code>	<code>x == y</code>	Is x equal to y?
<code>!=</code>	<code>x != y</code>	Is x not equal to y?
<code>></code>	<code>x > y</code>	Is x greater than y?
<code><</code>	<code>x < y</code>	Is x less than y?
<code>>=</code>	<code>x >= y</code>	Greater than or equal?
<code><=</code>	<code>x <= y</code>	Less than or equal?

7. Logical Comparisons

We use comparison operators inside `if` statements to control the path our program takes.

```
int a = 5, b = 3; if (a > b) { cout << "Condition Met: a is larger than b"
<< endl; } if (a = 5) { cout << "Check: a is exactly 5" << endl; }
```

Warning: Use `==` for comparison. Using `=` inside an `if` statement is a common bug that assigns a value instead of checking it!

8. Logical Operators

What if you need to check two things at once? Logical operators combine multiple boolean values.

Operator	Symbol	Description
AND	&&	True ONLY if both sides are true
OR		True if at least one side is true
NOT	!	Flips the truth (True → False)

9. Advanced Logic Flow

Note how ! can be used to check if something is **not** the case.

```
bool x = true, y = false; if (x && !y) { cout << "x is true AND y is NOT  
true." << endl; } if (x || y) { cout << "At least one is true." << endl; }
```

10. Precedence of Operators

C++ follows specific rules to decide which operation happens first. This is similar to PEMDAS/BODMAS in mathematics.

The Hierarchy (High to Low)

1. **()** - Parentheses (Always First)
2. **++, --** - Increment / Decrement
3. ***, /, %** - Multiplication, Division, Modulus
4. **+, -** - Addition, Subtraction
5. **<, <=, >, >=** - Relational test
6. **==, !=** - Equality test
7. **&&** - Logical AND
8. **||** - Logical OR
9. **=, +=, etc.** - Assignment (Last)

11. Precedence Mastery

In the example below, notice how parentheses can completely change the result of a calculation.

```
int x = 5, y = 2, z = 10; // 3 * 10 = 30, then 5 + 30 = 35 int result1 = x
+ y * z; // (5 + 2) = 7, then 7 * 10 = 70 int result2 = (x + y) * z; cout
<< result1 << " vs " << result2;
```

12. Bonus: Bitwise Operators

For advanced CodeHive developers, bitwise operators allow you to manipulate individual bits (0s and 1s) inside a variable.

- **& (Bitwise AND)**
- **| (Bitwise OR)**
- **^ (Bitwise XOR)**
- **~ (Bitwise NOT)**
- **<< (Left Shift)**

These are mostly used in system programming, cryptography, and game engine optimization.

13. Miscellaneous Operators

A few extra tools to help you code more efficiently.

- **sizeof():** Tells you the size of a data type in bytes.
- **Ternary (?:):** A one-line if-else statement.
- **Comma (,):** Used to separate multiple expressions in one line.

```
int a = 10; string res = (a > 5) ? "Greater" : "Smaller";
```

14. Code Challenges / Mini Exercises

Time to test your skills! Open your IDE and try to solve these CodeHive Lab challenges.

Challenge 1: Write a program to swap two numbers using only arithmetic operators (no temp variable).

Challenge 2: Find the largest of three numbers using only comparison operators.

Challenge 3: Write a program that takes one number and checks if it is divisible by both 2 and 3 using logical operators.

15. Final Summary Checklist

Before moving on, verify your understanding of these critical concepts:

- ☒ I can calculate remainders using %.
- ☒ I know that x++ increases value by 1.
- ☒ I understand the difference between = and ==.

- ☒ I can combine conditions using `&&` and `||`.
- ☒ I know that `( )` has the highest priority.

Mastered: C++ Operators

© 2026 CodeHive Academy. This instructional material is proprietary.